

GitHub Actions + Advanced Security

Guia de Estudio en Espanol — Todo lo que necesitas dominar

Preparacion para Techno Week 8.0 — Banco de Costa Rica

Esta guia cubre GitHub Actions y GitHub Advanced Security enfocada en lo que necesitas para tu charla: explicar como funciona el pipeline, responder preguntas tecnicas, y demostrar que dominas la plataforma.

1. Anatomia de un Workflow

Un workflow es un archivo YAML que vive en `.github/workflows/` y define un proceso automatizado. Se compone de 4 niveles jerarquicos:

NIVEL	QUE ES	ANALOGIA
Workflow	El archivo YAML completo	La receta de cocina
Job	Un grupo de pasos que corren en el mismo runner	Una estacion de la cocina
Step	Una accion individual dentro de un job	Un paso de la receta
Action	Un step reutilizable (propio o de terceros)	Un utensilio pre-fabricado

Estructura minima de un workflow:

```
name: "Mi Pipeline"           # Nombre visible en GitHub

on:                            # TRIGGER: cuando se ejecuta
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:                    # Permisos del GITHUB_TOKEN
  contents: read
  pull-requests: write

jobs:                           # Los JOBS que se ejecutan
  mi-job:                       # ID unico del job
    name: "Nombre visible"      # Nombre en la UI de GitHub
    runs-on: ubuntu-latest      # RUNNER: donde corre
    steps:                      # Los STEPS del job
      - uses: actions/checkout@v4 # Step 1: action de tercero
      - name: Run tests           # Step 2: comando propio
        run: npm test
```

Cada job corre en un runner independiente (una maquina virtual limpia). Los jobs corren en paralelo por defecto a menos que uses 'needs'.

1.1 Triggers (on:)

Los triggers definen CUANDO se ejecuta el workflow:

TRIGGER	CUANDO SE DISPARA	USO EN TU PIPELINE
push	Cuando se hace push a un branch	Correr en cada push a main
pull_request	Cuando se abre/actualiza un PR	Validar seguridad antes del merge
schedule (cron)	En un horario definido	Escaneo semanal con CodeQL
workflow_dispatch	Manual desde la UI	Correr pipeline a demanda
workflow_call	Llamado por otro workflow	Reusable workflows

```
# Ejemplo: multiples triggers
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
```

```
schedule:
  - cron: '0 6 * * 1'      # Lunes 6am UTC
workflow_dispatch:         # Boton manual en GitHub
```

1.2 Runners (runs-on:)

Un runner es la maquina donde corre el job. GitHub ofrece runners hospedados (gratis para repos publicos) o podes usar self-hosted runners (tu propia infra).

RUNNER	SO	CUANDO USARLO
ubuntu-latest	Ubuntu 22.04+	La mayoria de casos (tu pipeline)
windows-latest	Windows Server	Builds de .NET, PowerShell
macos-latest	macOS	Builds de iOS, Swift
self-hosted	Tu infra	Acceso a red interna, GPU, compliance

Para banca: si el codigo no puede salir del banco, se usan self-hosted runners dentro de la red corporativa.

2. Secrets y Environment Variables

2.1 Secrets

Los secrets son valores encriptados que nunca se exponen en logs. Se configuran en Settings > Secrets and variables > Actions. Hay 3 niveles:

NIVEL	ALCANCE	CASO DE USO
Repository	Solo ese repo	API keys específicas del proyecto
Organization	Todos los repos de la org	Credenciales compartidas (SonarQube, etc.)
Environment	Solo para un environment específico	Credenciales de producción vs staging

```
# Usar un secret en un step
steps:
  - name: Deploy
    run: ./deploy.sh
    env:
      API_KEY: ${ secrets.DEPLOY_API_KEY }
      DB_PASSWORD: ${ secrets.DB_PASSWORD }

# GITHUB_TOKEN es especial: se genera automáticamente
# No necesitas crearlo. Tiene permisos configurables.
  - name: Post comment
    uses: marocchino/sticky-pull-request-comment@v2
    with:
      path: report.md
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

GITHUB_TOKEN se genera automáticamente en cada workflow run. Sus permisos se definen con la sección 'permissions:' al inicio del workflow. NUNCA hardcodear secrets en el YAML.

2.2 Variables de entorno

```
# Variables a nivel de workflow (todos los jobs las ven)
env:
  NODE_VERSION: "20"
  API_URL: "http://localhost:3000"

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      CI: true
      # Variables a nivel de job
    steps:
      - name: Run tests
        env:
          NODE_ENV: test
          # Variables a nivel de step
        run: npm test
```

Variables se configuran también en Settings > Variables > Actions. A diferencia de secrets, las variables NO son encriptadas y son visibles en logs.

3. Encadenar Jobs con needs:

Por defecto, los jobs corren en PARALELO. Con 'needs' defines dependencias: un job espera a que otro termine antes de empezar.

```
jobs:
  # Estos 3 corren EN PARALELO (no tienen needs)
  spec-lint:
    runs-on: ubuntu-latest
    steps: [...]

  sast:
    runs-on: ubuntu-latest
    steps: [...]

  secrets:
    runs-on: ubuntu-latest
    steps: [...]

  # Este corre DESPUES de spec-lint (depende de el)
  dast:
    runs-on: ubuntu-latest
    needs: [spec-lint]          # Espera a que spec-lint termine
    steps: [...]

  # Este corre AL FINAL (depende de todos)
  compliance-report:
    runs-on: ubuntu-latest
    needs: [spec-lint, sast, secrets, dast]
    if: always()                # Corre aunque alguno falle
    steps: [...]
```

En tu pipeline: Spectral, Semgrep, Gitleaks y npm audit corren en paralelo (rapido). ZAP espera a Spectral (no tiene sentido atacar si la spec es invalida). El reporte espera a todos.

Acceder al resultado de un job anterior:

```
report:
  needs: [lint, test, security]
  if: always()
  steps:
    - run: |
        echo "Lint: ${ needs.lint.result }"          # success/failure/skipped
        echo "Test: ${ needs.test.result }"
        echo "Security: ${ needs.security.result }"
        # Usar esto para generar el reporte de compliance
```

4. Usar Actions de Terceros

Las actions son steps reutilizables creados por la comunidad o por GitHub. Se referencian con 'uses:' seguido de org/repo@version.

```
steps:
  # Action oficial de GitHub - checkout del codigo
  - uses: actions/checkout@v4

  # Action oficial - setup de Node.js con cache
  - uses: actions/setup-node@v4
    with:
      node-version: "20"
      cache: "npm"

  # Action de tercero - Spectral linting
  - uses: stoplightio/spectral-action@latest
    with:
      file_glob: "openapi.yaml"

  # Action de tercero - Gitleaks
  - uses: gitleaks/gitleaks-action@v2
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

  # Action de tercero - OWASP ZAP
  - uses: zaproxy/action-api-scan@v0.9.0
    with:
      target: "http://localhost:3000/api/v1"
      format: openapi
```

Anatomia de una referencia de action:

PARTE	EJEMPLO	SIGNIFICADO
org/repo	actions/checkout	Repositorio donde vive la action
@version	@v4	Version (tag, branch, o SHA)
with:	node-version: 20	Parametros de entrada
env:	GITHUB_TOKEN: ...	Variables de entorno

Seguridad: para produccion, pinear actions a un SHA especifico en vez de un tag. Los tags son mutables (alguien podria cambiar lo que apunta v4). Ejemplo: uses: actions/checkout@8ade135a41bc03ea155e62e844d188df1ea18608

5. Correr Contenedores en GitHub Actions

Podes correr un job completo dentro de un contenedor Docker, o usar services (contenedores auxiliares como bases de datos). Esto es clave para ZAP y Semgrep.

Job completo en contenedor:

```
sast:
  runs-on: ubuntu-latest
  container:
    image: semgrep/semgrep      # Todo el job corre dentro de este contenedor
  steps:
    - uses: actions/checkout@v4
    - run: semgrep ci --config auto
```

Contenedor como service (auxiliar):

```
integration-tests:
  runs-on: ubuntu-latest
  services:
    postgres:                # Base de datos para tests
      image: postgres:15
      env:
        POSTGRES_PASSWORD: test
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
  steps:
    - uses: actions/checkout@v4
    - run: npm test
    env:
      DATABASE_URL: postgresql://postgres:test@localhost:5432
```

ZAP como action con Docker:

```
# ZAP no corre como 'container:' sino como action que usa Docker internamente
dast:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: |
        npm ci && npm start &      # Levantar API
        sleep 5
    - uses: zaproxy/action-api-scan@v0.9.0
    with:
      target: "http://localhost:3000/api/v1"
      docker_name: "ghcr.io/zaproxy/zaproxy:stable" # Imagen Docker de ZAP
```

En tu pipeline: Semgrep corre como container (el job entero dentro del contenedor). ZAP corre como una action que internamente levanta su propio Docker. Son dos patrones distintos.

6. Postear Comentarios en PRs Automaticamente

El reporte de compliance de tu pipeline se postea como comentario en el PR. Esto es lo que ve el reviewer y lo que ve un auditor de SUGEF.

```
compliance-report:
  runs-on: ubuntu-latest
  needs: [spec-lint, sast, secrets, deps, dast]
  if: always() && github.event_name == 'pull_request'
  steps:
    - uses: actions/checkout@v4

    - name: Generate report
      run: |
        # Crear el reporte en Markdown
        echo "## DevSecOps Report" > report.md
        echo "| Check | Status |" >> report.md
        echo "|-----|-----|" >> report.md
        echo "| Spec Lint | ${ needs.spec-lint.result } |" >> report.md
        echo "| SAST | ${ needs.sast.result } |" >> report.md

    # Postear como "sticky comment" (se actualiza, no duplica)
    - uses: marocchino/sticky-pull-request-comment@v2
      with:
        path: report.md
```

sticky-pull-request-comment actualiza el mismo comentario en vez de crear uno nuevo cada vez que el pipeline corre. Así el PR tiene un solo reporte, siempre actualizado.

Otras formas de interactuar con PRs:

ACTION	QUE HACE
marocchino/sticky-pull-request-comment	Comentario que se actualiza (lo que usas)
actions/github-script	Ejecutar JavaScript con la API de GitHub
peter-evans/create-pull-request	Crear un PR automaticamente
hmarr/auto-approve-action	Auto-aprobar PRs que pasen checks

7. GitHub Advanced Security (GHAS)

GHAS es el conjunto de features de seguridad nativas de GitHub. Desde abril 2025 se vende en dos productos separados:

PRODUCTO	PRECIO	INCLUYE
Secret Protection	\$19/commiter/mes	Secret scanning, Push Protection, alertas
Code Security	\$30/commiter/mes	CodeQL, Copilot Autofix, Dependabot, dependency review
Repos publicos	Gratis	Todas las features de GHAS son gratis

7.1 CodeQL — Code Scanning

CodeQL es el motor de analisis semantico de GitHub. A diferencia de Semgrep que busca patrones, CodeQL entiende el FLUJO de datos: rastrea como un input del usuario viaja por el codigo hasta llegar a una query SQL o una respuesta HTTP. Esto le permite encontrar vulnerabilidades complejas como inyeccion SQL a traves de multiples funciones.

```
# .github/workflows/codeql.yml
name: CodeQL Analysis
on:
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 6 * * 1'          # Escaneo semanal

jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions:
      security-events: write    # Necesario para subir resultados
    steps:
      - uses: actions/checkout@v4

      - uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
          queries: security-and-quality # Suite mas completa

      - uses: github/codeql-action/autobuild@v3

      - uses: github/codeql-action/analyze@v3
```

CodeQL soporta: JavaScript/TypeScript, Python, Java/Kotlin, C#, C/C++, Go, Ruby, Swift. NO soporta PHP, Scala ni Rust.

7.2 Secret Scanning + Push Protection

Secret scanning busca patrones de secretos conocidos (API keys de AWS, tokens de GitHub, passwords de DB) en tu repositorio. Push Protection va un paso mas alla: BLOQUEA el push antes de que el secreto llegue al repo. El developer ve un error y debe remover el secreto.

Se activa en: Settings > Code security > Secret scanning > Enable. Push protection: misma seccion > Push protection > Enable. No requiere workflow YAML — es una feature del repo.

Diferencia con Gitleaks: Gitleaks corre en el pipeline y detecta DESPUES del push. Push Protection bloquea ANTES. Son complementarios: Push Protection para prevencion inmediata, Gitleaks para escaneo profundo

del historial.

7.3 Dependabot

Dependabot monitorea tus dependencias 24/7. Cuando se publica un CVE que afecta una de tus dependencias, automaticamente crea un PR con la actualizacion. Tambien puede crear PRs de actualizacion de versiones (no solo seguridad).

```
# .github/dependabot.yml (en la raiz del repo)
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"          # Revisar semanalmente
    open-pull-requests-limit: 10
    reviewers:
      - "security-team"          # Asignar al equipo de seguridad

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"          # Tambien actualiza las actions
```

Diferencia con npm audit: npm audit se ejecuta bajo demanda en el pipeline. Dependabot monitorea CONTINUAMENTE y crea PRs automaticos. npm audit te dice 'tenes un problema', Dependabot te dice 'tenes un problema Y aqui esta el fix listo para merge'.

8. Reusable Workflows (Estandarizacion)

Los reusable workflows permiten definir un pipeline UNA vez y reutilizarlo en multiples repositorios. Es la clave para la estandarizacion organizacional que pide gerencia.

El workflow reutilizable (en el repo central):

```
# org/.github/.github/workflows/devsecops.yml
name: Reusable DevSecOps Pipeline
on:
  workflow_call:          # Se puede llamar desde otros workflows
  inputs:
    spec-file:
      type: string
      default: "openapi.yaml"
    security-tier:
      type: string
      default: "standard"  # basic, standard, strict

jobs:
  spec-lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: stoplightio/spectral-action@latest
        with:
          file_glob: ${ inputs.spec-file }
  # ... mas jobs segun el tier
```

Cualquier repo lo invoca con una linea:

```
# En cualquier repo del banco: .github/workflows/security.yml
name: Security
on: [pull_request]
jobs:
  security:
    uses: org/.github/.github/workflows/devsecops.yml@main
    with:
      spec-file: openapi.yaml
      security-tier: strict  # Este repo maneja datos de clientes
    secrets: inherit        # Hereda secrets de la org
```

Si actualizas el workflow central, TODOS los repos reciben la actualizacion automaticamente en el proximo run. No hay que tocar cada repo individualmente.

9. Referencia Rapida para la Charla

Si preguntan 'Por que GitHub Actions y no Jenkins/GitLab CI?'

El concepto aplica igual en cualquier CI/CD. Use GitHub porque: esta integrado con el repo (cero config extra), el marketplace tiene 20,000+ actions reutilizables, y GHAS solo esta disponible en GitHub.

Si preguntan 'Cuanto cuesta correr el pipeline?'

Repos publicos: 100% gratis, minutos ilimitados. Repos privados: 2,000 minutos/mes gratis en el plan Free, 3,000 en Pro. El pipeline completo toma ~5 minutos por corrida.

Si preguntan 'Que pasa si GitHub se cae?'

Si GitHub Actions no esta disponible, no se puede hacer merge — que es exactamente el comportamiento deseado: si no puedo verificar seguridad, no puedo deployar. Para redundancia, se puede correr el pipeline localmente tambien.

Si preguntan 'Como se manejan los secrets en un banco?'

Los secrets viven en GitHub Secrets (encriptados con libsodium). Para bancos con requisitos estrictos: GitHub se integra con Azure Key Vault, AWS Secrets Manager, y HashiCorp Vault para rotacion automatica.

Si preguntan 'CodeQL vs Semgrep?'

No son excluyentes. Semgrep busca patrones (rapido, facil de configurar). CodeQL entiende flujo de datos (mas profundo, encuentra vulnerabilidades complejas). En el pipeline usamos ambos: Semgrep como primera linea, CodeQL como analisis profundo.

Si preguntan 'Push Protection vs Gitleaks?'

Push Protection bloquea ANTES del push (prevencion). Gitleaks escanea DESPUES (deteccion). Usar ambos: Push Protection como primera barrera, Gitleaks para escaneo del historial completo.

Fuentes: docs.github.com/en/actions | docs.github.com/en/code-security

Tiempo de estudio: 3 horas con esta guia + practicar con tu repo.